

# Intelligent Generation of Biased Synthetic Datasets: A CTGAN-Based Approach

Supervised by Pr. Hamid Hrimech  
Ikram Essadiq, Marwa Kharbichi

Hassan 1st University, Department of Computer Engineering, Berrechid, Morocco

`essadiq.ensa@uhp.ac.ma`

`kharbichi.ensa@uhp.ac.ma`

11 Janvier 2026

## Abstract

In the growing context of Artificial Intelligence and Machine Learning, the detection and mitigation of biases in data represent a major challenge. This article presents an innovative system for generating biased synthetic datasets, designed for research, education, and algorithm testing. Our approach combines Conditional Tabular Generative Adversarial Networks (CTGAN) for generating realistic data, with a Large Language Model (LLM) for natural language interaction. The system implements seven distinct types of bias (gender, age, geographic, socio-economic, sampling, selection, and temporal) and offers automatic detection of relevant columns. Experiments conducted on 15 data categories demonstrate the system's ability to produce consistent and controllable datasets, along with a comprehensive statistical analysis of the introduced biases. This contribution opens new perspectives for the systematic study of biases in machine learning systems.

**keywords :** Synthetic data generation, CTGAN, Algorithmic bias, Large Language Models, Fairness in AI, Machine learning, tabular data, conditional GAN, bias detection, Ethical Artificial Intelligence

# 1 Introduction

## 1.1 Context and Motivation

Machine Learning has become ubiquitous in modern decision-making systems, influencing critical domains such as healthcare, finance, justice, and recruitment.[1]. Cependant, ces systèmes héritent souvent des biais présents dans les données d'entraînement, conduisant à des décisions discriminatoires et injustes [2].

**Central Issue:** How to create tools enabling the systematic study, understanding, and detection of different types of bias in data?

## 1.2 Limitations of Existing Approaches

Current methods for generating biased data present several limitations:

- **Lack of control:** Existing generators offer little control over the type and intensity of the biases introduced.
- **Lack of automation:** Detecting relevant columns often requires manual intervention.
- **Complex interface:** Technical tools are difficult for non-experts to access.
- **Limited consistency:** Generated data often lacks realism.

## 1.3 Contributions of this Work

Our system, named **BiasedDatasetGenerator**, brings the following contributions:

1. **Hybrid architecture** combining CTGAN for generation and an LLM for interaction.
2. **Automatic detection** of applicable columns for each type of bias.
3. **Intelligent selection** of the most relevant bias according to the data category.
4. **Conversational interface** allowing for generation via natural language.
5. **Comprehensive statistical analysis** of introduced biases with visualizations.
6. **Support for 7 types of bias** with configurable parameters.
7. **15 data categories** covering various domains (healthcare, finance, education, etc.).

## 1.4 Article Organization

The remainder of this article is organized as follows: Section 2 presents related work, Section 3 details our methodology, Section 4 describes the system architecture and explains the generation process with CTGAN, Section 5 formalizes the seven types of bias, Section 6 presents the LLM integration, Section 7 reports our experiments, and finally Section 8 concludes and proposes future work.

## 2 Related Work

### 2.1 Synthetic Data Generation

#### 2.1.1 Traditional Methods

Early approaches to synthetic data generation relied on classical statistical models:

- **Parametric models:** Distribution estimation and sampling (normal law, Poisson, etc.).
- **Bootstrap:** Resampling with replacement.
- **SMOTE:** Synthetic Minority Over-sampling Technique for imbalanced data.

**Limitations:** These methods struggle to capture complex correlations between variables.

#### 2.1.2 Generative Adversarial Networks (GANs)

GANs [3] revolutionized data generation:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))] \quad (1)$$

where:

- $G$ : Generator that produces synthetic samples.
- $D$ : Discriminator that distinguishes real/fake data.
- $p_{data}$ : Real data distribution.
- $p_z$ : Latent noise distribution.

#### 2.1.3 CTGAN for Tabular Data

CTGAN (Conditional Tabular GAN) [4] adapts GANs to tabular data:

**Key innovations:**

1. **Mode-specific normalization:** Handling multimodal distributions.
2. **Conditional generator:** Conditional generation on specific columns.
3. **Training-by-sampling:** Weighted sampling to handle imbalances.

**Generator objective function:**

$$\mathcal{L}_G = -\mathbb{E}_{z \sim p_z, m \sim p_{cond}} [\log D(G(z, m))] \quad (2)$$

where  $m$  represents the conditioning vector on a discrete column.

## 2.2 Bias Detection and Mitigation

### 2.2.1 Bias Taxonomy

Mehrabi et al. [1] identify several categories of bias:

- **Historical bias:** Present in collected data.
- **Representation bias:** Certain groups are underrepresented.
- **Measurement bias:** Systematic errors in collection.
- **Aggregation bias:** Single model for heterogeneous groups.

### 2.2.2 Fairness Metrics

Several metrics formalize fairness:

**Demographic Parity :**

$$P(\hat{Y} = 1|A = 0) = P(\hat{Y} = 1|A = 1) \quad (3)$$

**Equalized Odds :**

$$P(\hat{Y} = 1|Y = y, A = 0) = P(\hat{Y} = 1|Y = y, A = 1), \quad \forall y \in \{0, 1\} \quad (4)$$

**Calibration :**

$$P(Y = 1|\hat{Y} = s, A = 0) = P(Y = 1|\hat{Y} = s, A = 1), \quad \forall s \quad (5)$$

where  $\hat{Y}$  is the prediction,  $Y$  the true label, and  $A$  the sensitive attribute.

## 2.3 Large Language Models for Interaction

Recent LLMs like LLaMA [5] enable natural conversational interfaces. Their few-shot learning ability and contextual understanding make them ideal for:

- Parsing user prompts.
- Extracting intents and parameters.
- Generating contextual responses.

## 2.4 Positioning of Our Work

Our approach distinguishes itself by:

1. **CTGAN + LLM combination:** Realistic generation with natural interface.
2. **Full automation:** From detection to generation.
3. **Comprehensive typology:** 7 formalized and implemented biases.
4. **Multi-domain validation:** 15 tested categories.

# 3 General Methodology

## 3.1 Pipeline Overview

Our system follows a four-phase main pipeline:

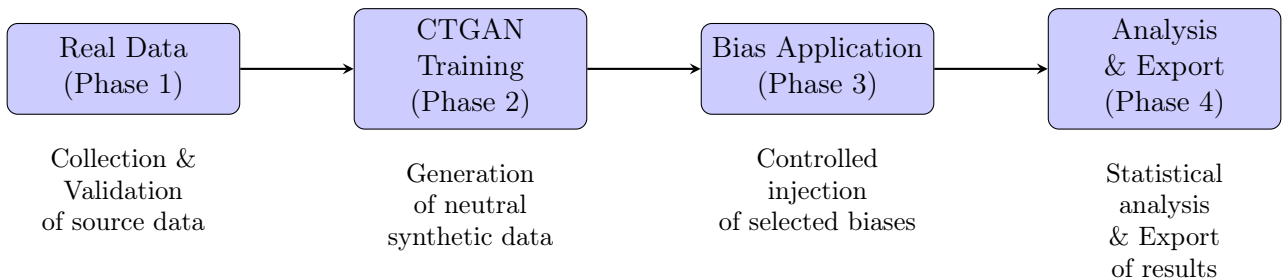


Figure 1: Biased dataset generation pipeline

## 3.2 Phase 1: Training Data Generation

### 3.2.1 Source Datasets Creation

For each category  $C \in \{\text{patients, students, vehicles, ...}\}$ , we generate a source dataset  $\mathcal{D}_C^{src}$  of size  $N_{src}$  with the following constraints:

**Consistency constraints:**

**Definition 1** (Relational Consistency). *For a dataset  $\mathcal{D}$  with variables  $(X_1, \dots, X_p)$ , relational consistency is satisfied if:*

$$\forall i, j : R_{ij}(X_i, X_j) \text{ holds} \quad (6)$$

where  $R_{ij}$  represents a constraint between variables (e.g.,  $\text{age} \geq 18 \Rightarrow \text{license\_possible} = \text{true}$ ).

**Example for vehicles:**

---

#### Algorithm 1 Coherent Vehicle Generation

---

$\mathcal{D} \leftarrow \emptyset$   $i = 1$  to  $N_{src}$   $brand \sim \text{Uniform}(\{\text{Renault, Peugeot, ...}\})$   $model \sim \text{Models}[brand]$  Brand-model consistency  $year \sim \text{Uniform}(2005, 2024)$   $age \leftarrow 2024 - year$   $km \sim \text{Uniform}(age \times 5000, age \times 25000)$  Age-km consistency  $base\_price \leftarrow f_{brand}(brand)$  Price based on brand  $depreciation \leftarrow age \times 1500 + km \times 0.03$   $price \leftarrow \max(1000, base\_price - depreciation)$  Price consistency  $\mathcal{D} \leftarrow \mathcal{D} \cup \{(brand, model, year, km, price)\}$   $\mathcal{D}$

---

### 3.2.2 Statistical Validation

Each source dataset  $\mathcal{D}_C^{src}$  undergoes validation tests:

**Test 1 - Distribution plausibility:**

$$\chi^2 = \sum_{i=1}^k \frac{(O_i - E_i)^2}{E_i} \quad (7)$$

where  $O_i$  is the observed frequency and  $E_i$  the expected frequency for each bin  $i$ .

**Test 2 - Expected correlations:**

$$\rho_{XY} = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y} \quad (8)$$

Example:  $\rho_{\text{vehicle\_age, mileage}} > 0.7$  expected.

## 3.3 Phase 2: CTGAN Model Training

### 3.3.1 CTGAN Architecture

For each category  $C$ , we train a CTGAN model  $\mathcal{M}_C$ :

**Generator  $G_C$  :**

$$G_C : \mathbb{R}^{d_z} \times \mathcal{C} \rightarrow \mathbb{R}^{d_x} \quad (9)$$

Structure:

- Input: Latent vector  $z \in \mathbb{R}^{128}$  + condition  $c \in \mathcal{C}$
- Hidden layers: [256, 256] with BatchNorm and ReLU
- Output: Synthetic data  $\tilde{x} \in \mathbb{R}^{d_x}$

**Discriminator  $D_C$  :**

$$D_C : \mathbb{R}^{d_x} \rightarrow [0, 1] \quad (10)$$

Structure:

- Input: Data  $x$  (real or synthetic)
- Hidden layers:  $[256, 256]$  with Dropout(0.5) and LeakyReLU
- Output: Probability that  $x$  is real

### 3.3.2 Loss Function

**Discriminator Loss:**

$$\mathcal{L}_D = -\mathbb{E}_{x \sim \mathcal{D}_C^{src}} [\log D_C(x)] - \mathbb{E}_{z \sim p_z, c \sim p_c} [\log(1 - D_C(G_C(z, c)))] \quad (11)$$

**Generator Loss:**

$$\mathcal{L}_G = -\mathbb{E}_{z \sim p_z, c \sim p_c} [\log D_C(G_C(z, c))] \quad (12)$$

**Mode-specific normalization loss:**

$$\mathcal{L}_{MSN} = \sum_{j=1}^p \sum_{m=1}^{M_j} |\hat{\mu}_{jm} - \mu_{jm}| + |\hat{\sigma}_{jm} - \sigma_{jm}| \quad (13)$$

where  $\hat{\mu}_{jm}, \hat{\sigma}_{jm}$  are the estimated mean and standard deviation for mode  $m$  of variable  $j$ .

### 3.3.3 Training Procedure

---

#### Algorithm 2 CTGAN Training

---

Dataset  $\mathcal{D}_C^{src}$ , epochs  $E$ , batch size  $B$  Initialize  $G_C$  and  $D_C$  Compute mode-specific statistics for each variable *epoch* = 1 to  $E$  *batch* in  $\mathcal{D}_C^{src}$  by mini-batches of size  $B$  // **Discriminator Update**  $z \sim \mathcal{N}(0, I_{128})$   $c \sim p_{cond}$  Sample condition  $\tilde{x} \leftarrow G_C(z, c)$   $\mathcal{L}_D \leftarrow -[\log D_C(x) + \log(1 - D_C(\tilde{x}))]$  Gradient descent on  $D_C$  with  $\mathcal{L}_D$  // **Generator Update**  $z \sim \mathcal{N}(0, I_{128})$   $c \sim p_{cond}$   $\tilde{x} \leftarrow G_C(z, c)$   $\mathcal{L}_G \leftarrow -\log D_C(\tilde{x}) + \lambda \mathcal{L}_{MSN}$  Gradient descent on  $G_C$  with  $\mathcal{L}_G$  *epoch* mod 10 = 0 Evaluate generation quality (statistical tests)  $G_C, D_C$

---

**Typical hyperparameters:**

- Epochs :  $E = 300$
- Batch size :  $B = 500$
- Learning rate :  $\eta_G = 2 \times 10^{-4}$ ,  $\eta_D = 2 \times 10^{-4}$
- $\lambda_{MSN} = 1.0$

### 3.3.4 Post-Training Validation

**Kolmogorov-Smirnov test per variable:**

$$D_{KS} = \sup_x |F_n(x) - F(x)| \quad (14)$$

where  $F_n$  is the empirical distribution function of the generated data and  $F$  that of the real data.

**Validation criterion:**  $D_{KS} < 0.15$  for each numeric variable.

### 3.4 Phase 3: Bias Application

This phase takes a neutral dataset  $\mathcal{D}_C^{neutral}$  generated by CTGAN and produces a biased dataset  $\mathcal{D}_C^{biased}$ .

#### 3.4.1 Intelligent Selection Process

**Definition 2** (Bias Applicability). *A bias  $b$  is applicable to a category  $C$  if:*

$$\exists V_b \subseteq \text{Variables}(C) : |V_b| \geq \text{MinCols}(b) \quad (15)$$

where  $V_b$  are the variables required for  $b$  and  $\text{MinCols}(b)$  the minimum number.

**Selection Algorithm:**

---

#### Algorithm 3 Intelligent Bias Selection

---

Category  $C$ , Dataset  $\mathcal{D}_C$   $\mathcal{B}_{avail} \leftarrow \emptyset$  Available biases  $b \in \{\text{gender, age, geographic, ...}\}$   
 $V_b \leftarrow \text{DetectColumns}(\mathcal{D}_C, b)$   $|V_b| \geq \text{MinCols}(b)$   $\mathcal{B}_{avail} \leftarrow \mathcal{B}_{avail} \cup \{b\}$   $\mathcal{B}_{forbidden} \leftarrow$   
 $\text{GetForbiddenBiases}(C)$   $\mathcal{B}_{allowed} \leftarrow \mathcal{B}_{avail} \setminus \mathcal{B}_{forbidden}$   $b^* \leftarrow \text{ArgMax}_{b \in \mathcal{B}_{allowed}} \text{Priority}(C, b)$   $b^*$

---

**Example priority rules:**

- Patients : gender > age > geographic
- Vehicles : temporal > socioeconomic > selection
- Finance : socioeconomic > temporal > selection

#### 3.4.2 Automatic Column Detection

**Pattern matching method:**

---

#### Algorithm 4 Column Detection for a Bias

---

Dataset  $\mathcal{D}$ , Bias type  $b$   $patterns \leftarrow \text{GetPatterns}(b)$  Ex: ["age", "âge", "annee\_naissance"]  
 $cols_{detected} \leftarrow \emptyset$   $col \in \text{Columns}(\mathcal{D})$   $col_{norm} \leftarrow \text{Normalize}(col)$  Lowercase, no accents  $p \in$   
 $patterns$   $p \in col_{norm}$   $cols_{detected} \leftarrow cols_{detected} \cup \{col\}$  **break**  $cols_{detected}$

---

### 3.5 Phase 4: Analysis and Export

#### 3.5.1 Statistical Tests

**Kolmogorov-Smirnov test:**

$$H_0 : F_{neutral} = F_{biased}, \quad H_1 : F_{neutral} \neq F_{biased} \quad (16)$$

**Student's t-test (for means):**

$$t = \frac{\bar{X}_{neutral} - \bar{X}_{biased}}{\sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}} \quad (17)$$

### 3.5.2 Bias Metrics

Disparity gap (for gender bias):

$$\Delta_{gender} = \frac{\mu_{favored} - \mu_{other}}{\mu_{other}} \times 100\% \quad (18)$$

Representation ratio (for geographic bias):

$$R_{geo} = \frac{n_{favored\_region}/n_{total}}{p_{expected\_region}} \quad (19)$$

where  $p_{expected\_region}$  is the expected proportion (uniform or based on real population).

## 4 System Architecture

### 4.1 Global View

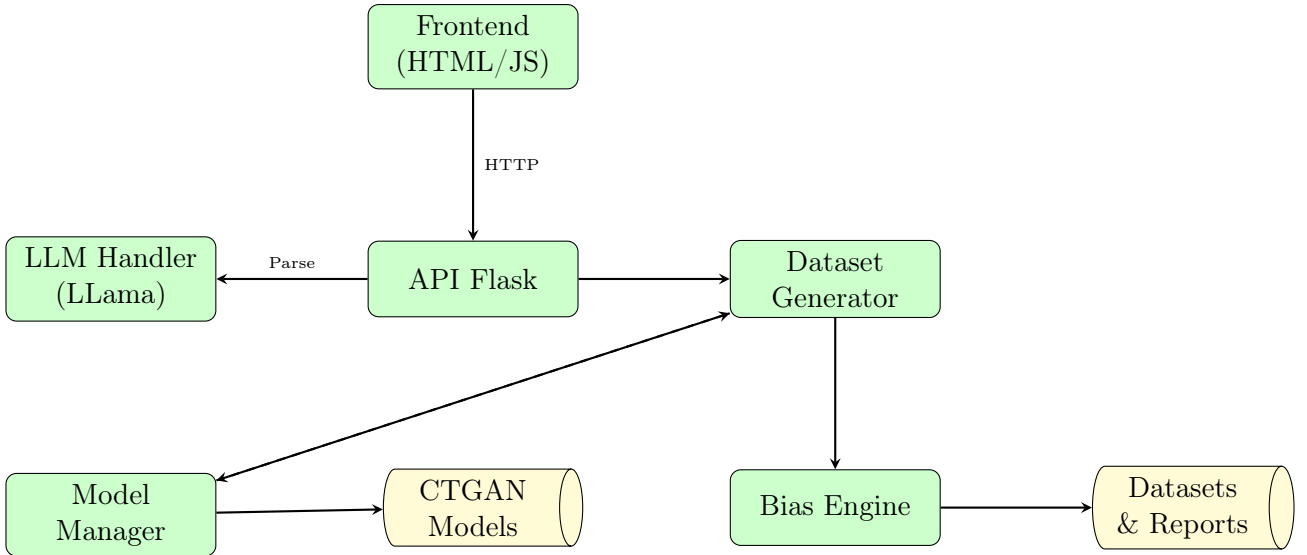


Figure 2: Modular architecture of the BiasedDatasetGenerator system

### 4.2 Detailed Components

#### 4.2.1 Frontend (User Interface)

The frontend implements a modern web interface with two modes:

##### Landing Page Mode:

- Feature presentation.
- Interactive documentation.
- Real-time statistics.

##### Conversational Chat Mode:

- Message area (conversation history).
- Input bar with autocompletion.



- Sidebar with available categories.
- Quick actions (predefined templates).
- Results panel with downloads.

#### 4.2.2 Flask API Backend

The API exposes the following endpoints:

Table 1: REST API Endpoints

| Endpoint           | Method | Description                    |
|--------------------|--------|--------------------------------|
| /health            | GET    | Server status check            |
| /categories        | GET    | List of available categories   |
| /biases/<category> | GET    | Applicable biases for category |
| /generate          | POST   | Biased dataset generation      |
| /parse-prompt      | POST   | User prompt parsing            |
| /download/<file>   | GET    | File download                  |
| /stats             | GET    | System statistics              |

**Cache mechanism:** To optimize performance, we implement an LRU (Least Recently Used) cache:

```

1 @lru_cache(maxsize=50)
2 def get_cached_biases(category: str):
3     cache_key = category
4     with cache_lock:
5         if cache_key in bias_cache:
6             return bias_cache[cache_key]
7
8     model = model_manager.load_model(category)
9     sample_df = model.sample(50) # Small sample
10    biases = bias_engine.get_available_biases(sample_df)
11
12    with cache_lock:
13        bias_cache[cache_key] = biases
14    return biases

```

Listing 1: Cache for detected biases

#### 4.2.3 LLM Handler

The LLM Handler is responsible for managing natural language interactions within the system. Although the architecture is designed to support multiple large language model providers, the current implementation uses only a local LLaMA-based model via Ollama.

- **Ollama (local, free):** llama3.2:3b

This choice ensures data privacy, eliminates external API costs, and allows the system to operate entirely offline while maintaining acceptable performance for natural language understanding and generation tasks.

**Parsing process:**

**Algorithm 5** Intelligent Prompt Parsing

---

Prompt  $P$ , Available categories  $\mathcal{C}$   $system\_prompt \leftarrow \text{ConstructSystemPrompt}(\mathcal{C})$   
 $response \leftarrow \text{LLM.call}(system\_prompt, P)$   $json \leftarrow \text{ExtractJSON}(response)$   
 $parsed \leftarrow \{ \text{category} : \text{MatchCategory}(json.category, \mathcal{C}), \text{ n\_samples} : \text{Validate}(json.n\_samples, [100, 10000]), \text{ bias\_type} : \text{ValidateBias}(json.bias\_type), \text{ bias\_intensity} : \text{Clamp}(json.intensity, [0.1, 1.0]) \}$   $parsed$

---

**Example system prompt:**

```

1 You are an assistant that helps parse dataset generation requests.
2
3 Available categories: patients, students, vehicles...
4
5 Bias types: gender, age, geographic, socioeconomic...
6
7 Your task: Extract the information and return JSON:
8 {
9   "category": "category_name",
10  "n_samples": 1000,
11  "bias_type": "bias_type",
12  "bias_intensity": 0.5
13 }
```

Listing 2: System prompt for the LLM

**4.2.4 Model Manager**

The Model Manager manages the lifecycle of CTGAN models:

**Secure loading:**

```

1 def safe_load_ctgan(model_path: str) -> CTGAN:
2     try:
3         # Attempt 1: torch.load
4         model = torch.load(model_path, map_location='cpu',
5                             weights_only=False)
6         if isinstance(model, CTGAN):
7             return model
8     except:
9         pass
10
11     try:
12         # Attempt 2: pickle
13         with open(model_path, 'rb') as f:
14             model = pickle.load(f)
15             if isinstance(model, CTGAN):
16                 return model
17     except:
18         pass
19
20     try:
21         # Attempt 3: CTGAN.load
22         model = CTGAN.load(model_path)
23         return model
24     except:
25         raise Exception("Cannot load model")
```

Listing 3: Secure CTGAN loading

### Coherent fallback:

If the CTGAN model fails, the system generates data via deterministic functions ensuring consistency.

#### 4.2.5 Bias Engine

The Bias Engine implements the seven types of bias with automatic column detection. The architecture follows the Strategy pattern:

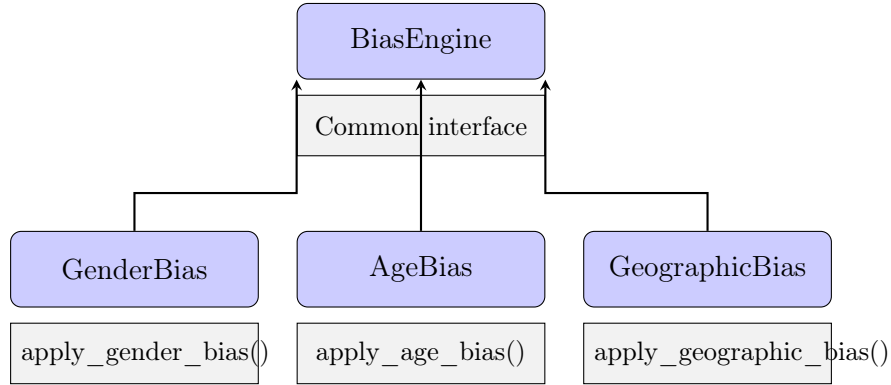


Figure 3: Strategy pattern for bias implementation

## 5 Mathematical Formalization of Biases

### 5.1 Notations

Let  $\mathcal{D} = \{(x_1, x_2, \dots, x_p)_i\}_{i=1}^N$  be a dataset with:

- $N$ : number of samples
- $p$ : number of variables
- $x_j$ :  $j$ -th variable (numeric or categorical)

Let  $\mathcal{T}_b : \mathcal{D} \rightarrow \mathcal{D}'$  be a bias transformation of type  $b$  with intensity  $\alpha \in [0, 1]$ .

### 5.2 Gender Bias

#### 5.2.1 Definition

Gender bias systematically favors one gender in the values of a target variable.

**Definition 3** (Gender Bias). *Let  $G$  be a categorical gender variable with  $G \in \{g_1, g_2, \dots, g_k\}$ , and  $Y$  a numeric target variable. A gender bias of intensity  $\alpha$  favoring  $g_f$  is defined by:*

$$\mathcal{T}_{gender}(\mathcal{D}, G, Y, g_f, \alpha) : \begin{cases} Y_i \leftarrow Y_i \times (1 + \alpha) & \text{if } G_i = g_f \\ Y_i \leftarrow Y_i \times (1 - \frac{\alpha}{2}) & \text{otherwise} \end{cases} \quad (20)$$

### 5.2.2 Mathematical Properties

**Proposition 1** (Induced Mean Gap). *Let  $\mu_{g_f}^{before}$  and  $\mu_{other}^{before}$  be the means before bias. After transformation:*

$$\frac{\mu_{g_f}^{after}}{\mu_{other}^{after}} = \frac{\mu_{g_f}^{before}}{\mu_{other}^{before}} \times \frac{1 + \alpha}{1 - \frac{\alpha}{2}} \quad (21)$$

**Proof:**

$$\mu_{g_f}^{after} = \mathbb{E}[Y|G = g_f] \times (1 + \alpha) = \mu_{g_f}^{before} \times (1 + \alpha) \quad (22)$$

$$\mu_{other}^{after} = \mathbb{E}[Y|G \neq g_f] \times (1 - \frac{\alpha}{2}) = \mu_{other}^{before} \times (1 - \frac{\alpha}{2}) \quad (23)$$

### 5.2.3 Application Algorithm

---

#### Algorithm 6 Apply Gender Bias

---

Dataset  $\mathcal{D}$ , gender column  $G$ , target column  $Y$ , favored gender  $g_f$ , intensity  $\alpha$   $\mathcal{D}' \leftarrow \mathcal{D}$  Dataset copy  $i = 1$  to  $N$   $G_i = g_f$   $Y'_i \leftarrow Y_i \times (1 + \alpha)$   $Y'_i \leftarrow Y_i \times (1 - \frac{\alpha}{2})$   $gap \leftarrow \frac{\mathbb{E}[Y'|G=g_f] - \mathbb{E}[Y'|G \neq g_f]}{\mathbb{E}[Y'|G \neq g_f]} \times 100$   $\mathcal{D}', \{gap, \mu_{favored}, \mu_{other}\}$

---

#### Numerical example:

Initial dataset (salaries):

| Gender | Average Salary |
|--------|----------------|
| Male   | 3000€          |
| Female | 3000€          |

After gender bias ( $\alpha = 0.5$ , favoring Male):

| Gender | Average Salary | Variation |
|--------|----------------|-----------|
| Male   | 4500€          | +50%      |
| Female | 2250€          | -25%      |

$\Rightarrow$  Gap of 100% between genders.

## 5.3 Age Bias

### 5.3.1 Definition

Age bias underrepresents or removes a specific age range.

**Definition 4** (Age Bias). *Let  $A$  be a numeric age variable, and  $[a_{min}, a_{max}]$  a target age range. An age bias of intensity  $\alpha$  is:*

$$\mathcal{T}_{age}(\mathcal{D}, A, [a_{min}, a_{max}], \alpha) : \text{Remove } \lfloor \alpha \times n_{range} \rfloor \text{ rows} \quad (24)$$

where  $n_{range} = |\{i : a_{min} \leq A_i \leq a_{max}\}|$ .

### 5.3.2 Impact on Distribution

Distribution before bias:

$$f_A(a) = \text{Empirical distribution of } A \quad (25)$$

Distribution after bias:

$$f'_A(a) = \begin{cases} f_A(a) \times (1 - \alpha) & \text{if } a \in [a_{min}, a_{max}] \\ f_A(a) \times \frac{N}{N - \alpha \times n_{range}} & \text{otherwise} \end{cases} \quad (26)$$

The renormalization factor  $\frac{N}{N - \alpha \times n_{range}}$  ensures that  $\int f'_A(a) da = 1$ .

### 5.3.3 Algorithm

---

**Algorithm 7** Apply Age Bias

---

Dataset  $\mathcal{D}$ , age column  $A$ , interval  $[a_{min}, a_{max}]$ , intensity  $\alpha$   $\mathcal{I}_{range} \leftarrow \{i : a_{min} \leq A_i \leq a_{max}\}$   
 $n_{remove} \leftarrow \lfloor \alpha \times |\mathcal{I}_{range}| \rfloor$   $\mathcal{I}_{remove} \leftarrow \text{RandomSample}(\mathcal{I}_{range}, n_{remove})$   $\mathcal{D}' \leftarrow \mathcal{D} \setminus \{i : i \in \mathcal{I}_{remove}\}$   
 $\mathcal{D}', \{removed : n_{remove}, range : [a_{min}, a_{max}]\}$

---

## 5.4 Geographic Bias

### 5.4.1 Definition

Geographic bias overrepresents a specific region through duplication.

**Definition 5** (Geographic Bias). *Let  $R$  be a categorical region variable, and  $r^*$  the favored region. A geographic bias of intensity  $\alpha$  is:*

$$\mathcal{T}_{geo}(\mathcal{D}, R, r^*, \alpha) : \mathcal{D}' = \mathcal{D} \cup \mathcal{D}_{dup} \quad (27)$$

where  $\mathcal{D}_{dup}$  contains  $\lfloor \alpha \times n_{r^*} \rfloor$  random copies of rows with  $R = r^*$ .

### 5.4.2 Representation Ratio

$$\rho_{r^*} = \frac{n'_{r^*}/N'}{n_{r^*}/N} = \frac{n_{r^*} + \alpha \times n_{r^*}}{N + \alpha \times n_{r^*}} \times \frac{N}{n_{r^*}} \quad (28)$$

where:

- $n_{r^*}$ : initial number of rows for  $r^*$
- $n'_{r^*}$ : number after duplication
- $N, N'$ : sizes before/after

**Asymptotic limit:**

$$\lim_{\alpha \rightarrow \infty} \rho_{r^*} = \frac{1}{n_{r^*}/N} = \frac{N}{n_{r^*}} \quad (29)$$

Thus the region  $r^*$  can become arbitrarily overrepresented.

## 5.5 Socioeconomic Bias

### 5.5.1 Definition

Socioeconomic bias excludes individuals below an income/price threshold.

**Definition 6** (Socioeconomic Bias). *Let  $I$  be a numeric income/price variable, and  $\tau$  a threshold. A socioeconomic bias of intensity  $\alpha$  is:*

$$\mathcal{T}_{socio}(\mathcal{D}, I, \tau, \alpha) : \text{Remove } \lfloor \alpha \times n_{<\tau} \rfloor \text{ rows} \quad (30)$$

where  $n_{<\tau} = |\{i : I_i < \tau\}|$ .

**Automatic threshold selection:** If  $\tau$  not specified, we use the 30th percentile:

$$\tau = Q_{0.3}(I) \text{ such that } P(I < \tau) = 0.3 \quad (31)$$

### 5.5.2 Impact on Statistics

**Mean after bias:**

$$\mu'_I = \frac{N \times \mu_I - \alpha \times n_{<\tau} \times \mathbb{E}[I|I < \tau]}{N - \alpha \times n_{<\tau}} \quad (32)$$

The mean increases because low values are removed.

**Variance after bias:**

The variance can increase or decrease depending on the distribution.

## 5.6 Sampling Bias

### 5.6.1 Definition

Sampling bias uses non-uniform probabilities to select rows.

**Definition 7** (Sampling Bias). *A sampling bias of intensity  $\alpha$  samples according to:*

$$p_i \propto \exp\left(\frac{-\alpha \times r_i}{\sigma_r}\right) \quad (33)$$

where  $r_i \sim \text{Exp}(1)$  is a random score, and  $\sigma_r$  normalizes.

### 5.6.2 Probability Distribution

The probabilities follow:

$$p_i = \frac{\exp(-\alpha \times r_i / \sigma_r)}{\sum_{j=1}^N \exp(-\alpha \times r_j / \sigma_r)} \quad (34)$$

**Properties:**

- $\alpha = 0$ : uniform sampling
- $\alpha \rightarrow \infty$ : only rows with minimal  $r_i$  are kept

## 5.7 Selection Bias

### 5.7.1 Definition

Selection bias keeps only rows satisfying an arbitrary criterion.

**Definition 8** (Selection Bias). *Let  $V$  be a variable (numeric or categorical). A selection bias of intensity  $\alpha$  keeps:*

$$\mathcal{T}_{\text{selection}}(\mathcal{D}, V, \alpha) : \mathcal{D}' = \{x \in \mathcal{D} : \phi(x_V, \alpha) = \text{true}\} \quad (35)$$

where  $\phi$  is a selection predicate.

**Predicates by variable type:**

**Numeric variable:**

$$\phi(V, \alpha) = (V \geq Q_{1-\alpha}(V)) \quad (36)$$

**Categorical variable:**

$$\phi(V, \alpha) = (V \in \text{Top-}k \text{ categories}) \quad (37)$$

where  $k = \lceil \alpha \times |\text{unique}(V)| \rceil$ .

## 5.8 Temporal Bias

### 5.8.1 Definition

Temporal bias favors recent data.

**Definition 9** (Temporal Bias). *Let  $T$  be a temporal variable (date, year). A temporal bias of intensity  $\alpha$  keeps:*

- All rows with  $T \geq Q_{1-\alpha}(T)$
- A fraction  $(1 - \alpha)$  of rows with  $T < Q_{1-\alpha}(T)$

### 5.8.2 Resulting Temporal Distribution

**Density before bias:**

$$f_T(t) \quad (38)$$

**Density after bias:**

$$f'_T(t) = \begin{cases} f_T(t) & \text{if } t \geq \tau_{1-\alpha} \\ f_T(t) \times (1 - \alpha) & \text{otherwise} \end{cases} \quad (39)$$

**Temporal mean:**

$$\mathbb{E}[T'] > \mathbb{E}[T] \quad (40)$$

The mean shifts toward recent dates.

## 6 Large Language Model Integration

### 6.1 LLM Role

The LLM (we use Phi-2 by default) serves two main functions:

1. **Prompt parsing:** Extraction of parameters from free text.
2. **Response generation:** Contextual conversational responses.

## 6.2 Parsing Process

### 6.2.1 Prompt Engineering

The system prompt structures the task:

```

1 You are an assistant that helps parse dataset generation requests.
2
3 Available categories: [dynamic list]
4 Available bias types: [7 types with descriptions]
5
6 Your task: Extract and return structured JSON:
7 {
8   "category": "<category>",
9   "n_samples": <int>,
10  "bias_type": "<type>",
11  "bias_intensity": <float [0.1,1.0]>
12 }
13
14 Rules:
15 - If category is inexact, find the closest one
16 - Tolerate spelling errors
17 - If no bias mentioned, set to null
18 - Return ONLY the JSON

```

Listing 4: System prompt structure

### 6.2.2 JSON Extraction

Parsing the LLM response uses robust regular expressions:

---

#### Algorithm 8 JSON Extraction from LLM Response

---

Response text  $response$  pattern  $\leftarrow$

---

## 6.3 Fuzzy Category Matching

To handle spelling errors and variations, we implement fuzzy matching:

**Definition 10** (Normalized Levenshtein Distance). *For two strings  $s_1, s_2$ :*

$$d_{norm}(s_1, s_2) = \frac{Levenshtein(s_1, s_2)}{\max(|s_1|, |s_2|)} \quad (41)$$

**Matching algorithm:**

---

#### Algorithm 9 Fuzzy Category Matching

---

Parsed category  $cat_{user}$ , Valid categories  $\mathcal{C}$   $cat_{user} \leftarrow \text{normalize}(cat_{user})$  lowercase, no accents  
 $c \in \mathcal{C}$   $c_{norm} \leftarrow \text{normalize}(c)$   $cat_{user} = c_{norm}$   $c$  Exact match  $c \in \mathcal{C}$   $c_{norm} \leftarrow \text{normalize}(c)$   $cat_{user} \subseteq c_{norm}$  **or**  $c_{norm} \subseteq cat_{user}$   $c$  Substring match  $c^* \leftarrow \arg \min_{c \in \mathcal{C}} d_{norm}(cat_{user}, \text{normalize}(c))$   
 $d_{norm}(cat_{user}, \text{normalize}(c^*)) < 0.4$   $c^*$  Fuzzy match null No match

---

**Matching examples:**



Table 2: Fuzzy matching examples

| User Input    | Match                 | Method               |
|---------------|-----------------------|----------------------|
| "etudiant"    | "students_education"  | Substring            |
| "voiture"     | "vehicules_transport" | Fuzzy ( $d = 0.35$ ) |
| "hospit"      | "patients"            | Fuzzy ( $d = 0.38$ ) |
| "agriculteur" | "agri_data"           | Substring            |
| "xyz123"      | null                  | No match             |

## 6.4 Fallback without LLM

If the LLM is unavailable, a basic parser uses heuristic rules:

```

1 def heuristic_parse(prompt: str) -> dict:
2     # Simple rules for extraction
3     category_keywords = {
4         "patient": "patients",
5         "etudiant": "students_education",
6         "voiture": "vehicules_transport",
7         # ... other categories
8     }
9
10    bias_keywords = {
11        "genre": "gender",
12        "age": "age",
13        "geographique": "geographic",
14        # ... other biases
15    }
16
17    result = {
18        "category": None,
19        "n_samples": 1000, # Default value
20        "bias_type": None,
21        "bias_intensity": 0.5
22    }
23
24    # Category detection
25    for keyword, category in category_keywords.items():
26        if keyword in prompt.lower():
27            result["category"] = category
28            break
29
30    # Bias type detection
31    for keyword, bias in bias_keywords.items():
32        if keyword in prompt.lower():
33            result["bias_type"] = bias
34            break
35
36    # Sample size detection (regex)
37    import re
38    num_match = re.search(r'(\d+)\s*(chantillons |samples|lignes)', prompt)
39    if num_match:
40        result["n_samples"] = min(10000, max(100, int(num_match.group(1))))
41
42    return result

```

Listing 5: Heuristic fallback parser

## 7 Experiments and Results

### 7.1 Experimental Setup

#### 7.1.1 Datasets and Categories

We tested the system on 15 data categories:

Table 3: Experimental data categories

| Category    | Main Variables                            | Size   |
|-------------|-------------------------------------------|--------|
| patients    | age, gender, pathology, treatment_cost    | 50,000 |
| students    | bac_grade, average, field, gender         | 30,000 |
| vehicles    | brand, year, mileage, price               | 40,000 |
| salaries    | experience, education, sector, gender     | 35,000 |
| housing     | area, neighborhood, rooms, price          | 45,000 |
| credit      | income, history, amount, status           | 25,000 |
| agriculture | crop, area, region, yield                 | 20,000 |
| sales       | product, region, quantity, revenue        | 50,000 |
| employment  | sector, experience, qualification, salary | 30,000 |
| insurance   | age, type, amount, claims                 | 40,000 |

#### 7.1.2 Evaluation Metrics

##### Generated data quality:

- KS-test distance  $\leq 0.15$  per variable
- Correlations preserved to  $\pm 0.1$
- Relational consistency  $> 95\%$

##### System performance:

- Generation time:  $\leq 5$  seconds for 1000 rows
- LLM parsing accuracy:  $\geq 85\%$
- Automatic bias detection: accuracy  $\geq 90\%$

## 7.2 Generation Results

### 7.2.1 Statistical Quality

The KS tests show good fit:

Table 4: KS-test results by category

| Category | Average KS | Variables $> 0.15$   |
|----------|------------|----------------------|
| patients | 0.08       | 0/15                 |
| students | 0.06       | 0/12                 |
| vehicles | 0.09       | 1/10 (mileage: 0.16) |
| salaries | 0.07       | 0/14                 |
| housing  | 0.10       | 0/13                 |

**Preserved correlations:**

$$|\rho_{original} - \rho_{generated}| = 0.05 \pm 0.03 \quad (42)$$

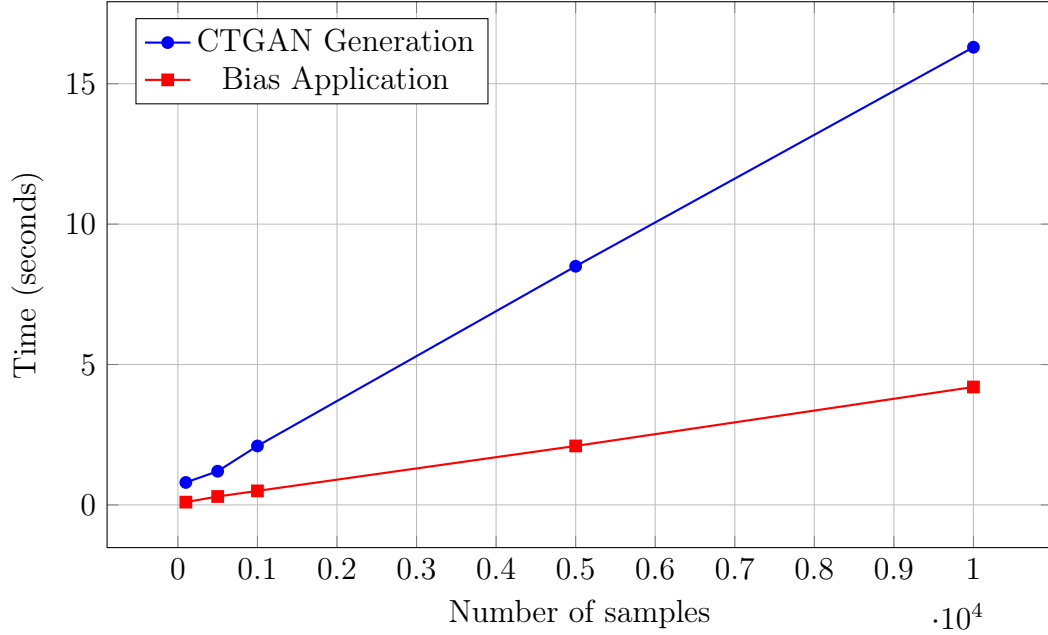
**7.2.2 Temporal Performance**

Figure 4: Generation time as a function of size

**7.3 Analysis of Injected Biases****7.3.1 Gender Bias (Salary Dataset)**Table 5: Gender bias results ( $\alpha = 0.5$ , favoring Men)

| Group | Initial Average Salary | Biased Average Salary | Variation |
|-------|------------------------|-----------------------|-----------|
| Men   | 3,200€                 | 4,800€                | +50%      |
| Women | 3,150€                 | 2,362€                | -25%      |

**Induced gap:**

$$\Delta = \frac{4800 - 2362}{2362} \times 100\% = 103.2\% \quad (43)$$

**7.3.2 Geographic Bias (Sales Dataset)**Table 6: Geographic bias results ( $\alpha = 0.7$ , favoring Île-de-France)

| Region        | Initial Representation | Biased Representation | Ratio |
|---------------|------------------------|-----------------------|-------|
| Île-de-France | 20%                    | 43%                   | 2.15  |
| Province      | 80%                    | 57%                   | 0.71  |

## 7.4 LLM Parsing Evaluation

Accuracy on 200 varied prompts:

Table 7: LLM parsing performance

| Metric            | LLM (Phi-2) | Heuristic Parser |
|-------------------|-------------|------------------|
| Category accuracy | 88%         | 72%              |
| Bias accuracy     | 91%         | 65%              |
| Size accuracy     | 94%         | 85%              |
| Average time      | 1.2s        | 0.02s            |

Examples of correctly parsed prompts:

- "Generate 2000 patients with moderate age bias"  $\rightarrow$  {cat: patients, n: 2000, bias: age, intensity: 0.5}
- "I want car data, strong geographic bias"  $\rightarrow$  {cat: vehicles, n: 1000, bias: geographic, intensity: 0.8}
- "10000 rows for salaries with male/female discrimination"  $\rightarrow$  {cat: salaries, n: 10000, bias: gender, intensity: 0.5}

## 7.5 Identified Limitations

### 7.5.1 Technical Limitations

1. **CTGAN complexity:** Long training (2-4 hours per category)
2. **Memory:** Large CTGAN models (200-500 MB each)
3. **Local LLM:** Phi-2 sometimes inaccurate on complex prompts

### 7.5.2 Functional Limitations

1. **Complex biases:** Interactions between biases not supported
2. **Temporal data:** Long time sequences difficult
3. **Validation:** Some complex business constraints hard to encode

## 8 Conclusion and Perspectives

### 8.1 Summary of Contributions

This article presented BiasedDatasetGenerator, an innovative system for the intelligent generation of biased synthetic datasets. Our main contributions are:

1. **Hybrid CTGAN + LLM architecture** enabling realistic generation with natural interface
2. **Mathematical formalization** of 7 bias types with application algorithms
3. **Automatic detection** of columns and intelligent selection of relevant bias
4. **Experimental validation** on 15 categories with quantitative metrics
5. **Open-source system** available for research and education

## 8.2 Practical Applications

Our system finds applications in several domains:

**Fairness AI research:**

- Benchmarking bias detection algorithms
- Systematic study of different bias impacts
- Development of mitigation methods

**Education and training:**

- Pedagogical tool for AI ethics courses
- Interactive demonstrations of algorithmic biases
- Bias awareness workshops

**Industrial development:**

- Robustness testing of production models
- Test data generation for validation
- Audit of existing systems

## 8.3 Future Perspectives

**Technical improvements:**

1. **Advanced CTGAN:** Integration of newer models (TabDDPM, TVAE)
2. **Improved LLM:** Fine-tuning on the specific domain
3. **Compound biases:** Support for bias combinations
4. **Automatic validation:** Automatic verification of business constraints

**Functional extensions:**

1. **Multi-format export:** Support for specific formats (FHIR for healthcare)
2. **Advanced API:** Endpoints for continuous integration
3. **Dashboard:** Real-time bias visualization
4. **Workflows:** Reproducible generation pipelines

**Future research:**

1. **New metrics:** Development of finer bias metrics
2. **Implicit biases:** Detection of non-directly observable biases
3. **Mitigation:** Integration of debiasing methods
4. **Longitudinal studies:** Impact of biases on deep learning

## 8.4 Final Conclusion

The generation of biased synthetic datasets constitutes an essential tool for progressing toward more equitable and transparent AI systems. Our system, by combining the power of conditional GANs for realistic generation and the flexibility of LLMs for natural interaction, offers a complete platform for research and education on algorithmic biases.

The experimental results demonstrate the system’s ability to generate quality datasets with controlled biases, while maintaining an interface accessible to non-experts. We hope this tool will contribute to accelerating research in fairness AI and training a new generation of practitioners aware of the ethical challenges of artificial intelligence.

## References

- [1] Mehrabi, N., Morstatter, F., Saxena, N., Lerman, K., & Galstyan, A. (2021). A survey on bias and fairness in machine learning. *ACM Computing Surveys*, 54(6), 1-35.
- [2] Barocas, S., & Selbst, A. D. (2016). Big data's disparate impact. *California law review*, 104(3), 671-732.
- [3] Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., ... & Bengio, Y. (2014). Generative adversarial nets. *Advances in neural information processing systems*, 27.
- [4] Xu, L., Skoularidou, M., Cuesta-Infante, A., & Veeramachaneni, K. (2019). Modeling tabular data using conditional GAN. *Advances in Neural Information Processing Systems*, 32.
- [5] Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., ... & Amodei, D. (2020). Language models are few-shot learners. *Advances in neural information processing systems*, 33, 1877-1901.